

PROGRESS UPDATE · HURRICANES HELENE & MILTON (2024)

Catching small damage in Treasure Island satellite imagery

How I followed up on the "even a single pixel matters" discussion: a bug fix, a combined signal, and a move to per-building analysis

Sai Vignesh Guturu · MS in Artificial Intelligence, Texas A&M

WHERE WE LEFT OFF

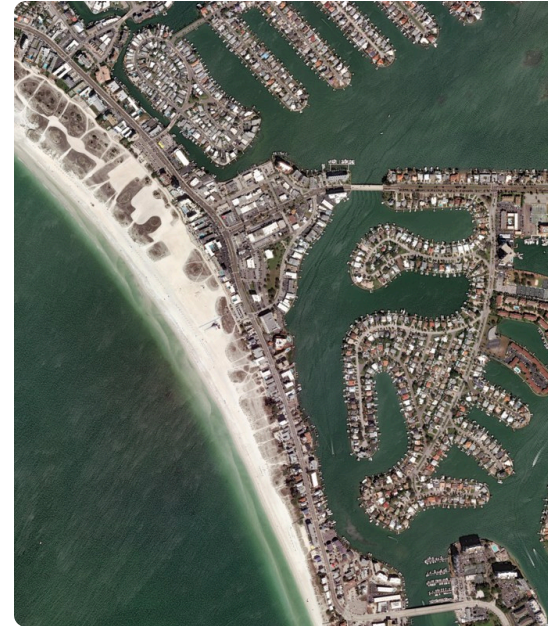
The question from our last discussion

The pipeline compares **before** and **after** satellite photos of Treasure Island, FL, and flags what changed.

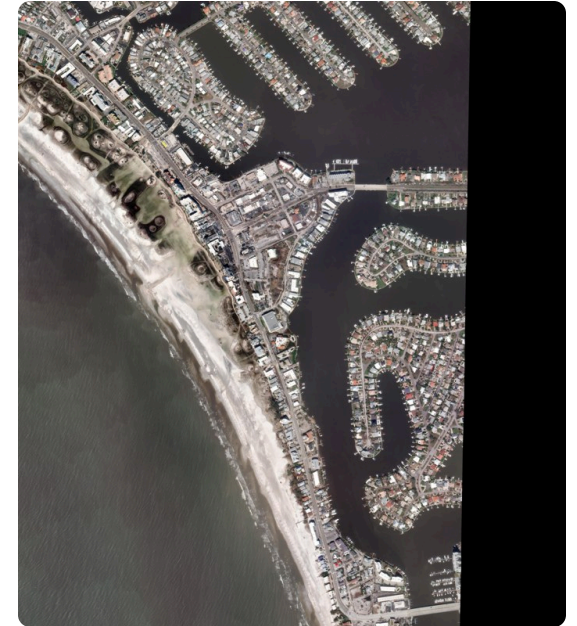
The concern raised after the call with Anwasha:

Would a very small, local change, even at the scale of a single pixel, show up at all?

When I looked into it, I found the honest answer was **"mostly no"**, and found out why.



Before: Maxar, Nov 2023



After: Maxar, Oct 2024 (post-Milton)

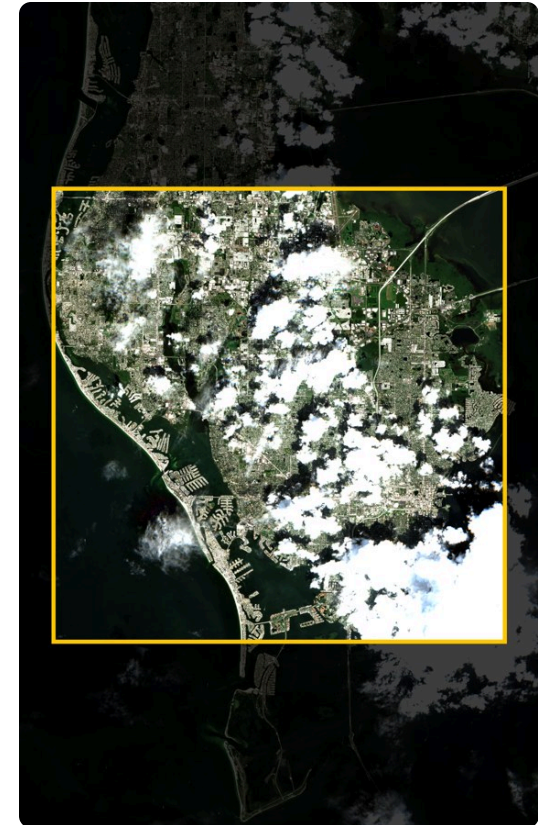
The AI model only saw the middle of each photo

DINOv2 (the AI model I use) comes with a default setting that **quietly crops every image to a centered square** before looking at it.

Anything outside that square is thrown away, so damage there **can never be detected**.

IMAGE PAIR	SIZE (W×H)	AREA ACTUALLY USED
Maxar	1402 × 1600	67.1%
NOAA	1600 × 1600	76.6%
Sentinel-2	1264 × 2000	48.4%

Up to ~half of the frame was never analyzed, and the tall Sentinel-2 image was hit hardest.



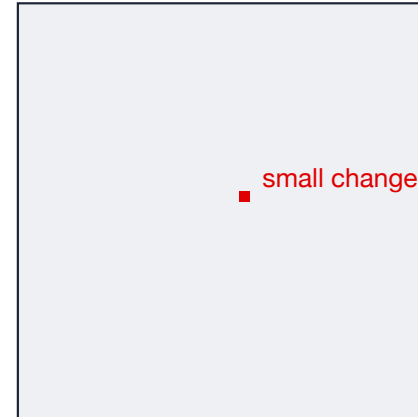
Sentinel-2: only the boxed area reached the model

...and what survived the crop was blurred

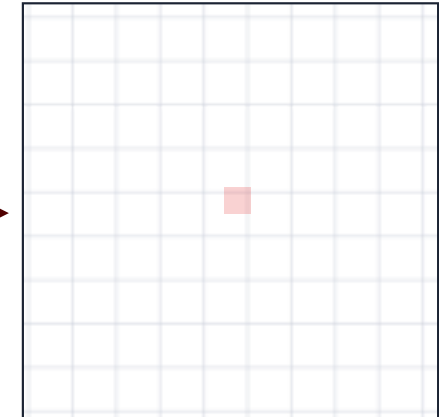
- 1 The cropped center was **shrunk** ~5× to 224×224 pixels.
- 2 The model then compares it in a coarse **16×16 grid** of blocks. Each block stood for roughly **70–100 original pixels** per side.
- 3 That 16×16 result was **stretched back** to full size, which smears any small signal across a large area.

Plain-language takeaway: a small, real change, like one damaged roof, was averaged into a large block and effectively disappeared.

Full photo (e.g. 1600 px)



16×16 comparison grid



1 block $\approx 100 \times 100$ original pixels.
A few changed pixels barely move its average $\rightarrow$ signal lost.

WHAT I CHANGED · 1

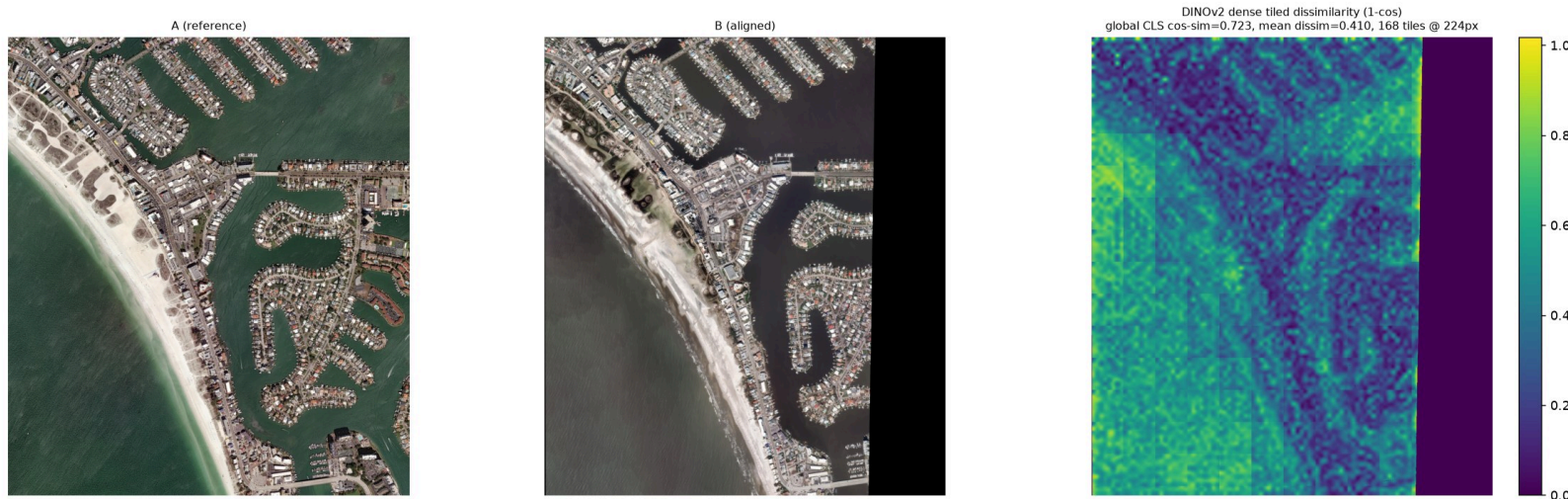
Fix: scan the whole photo in overlapping tiles

Turned off the hidden crop, so every pixel in the frame now counts.

Tiled the image into overlapping 224 px windows at original resolution (no shrinking), 168–196 tiles per pair.

Stitched tiles back into one full-resolution change map, averaging where tiles overlap.

Maxar: 2023-11-04 (pre) vs 2024-10-10 (post-Milton) — DINOv2 embedding similarity



Maxar before / after (aligned) / new change map. Brighter = more change. The map now covers the full frame and follows real structure (beach, canals, streets) instead of a blurry center patch. Whole run: ~40 s on a laptop.

Two detectors, and I trust where they agree

SSIM (classic, pixel-level)

Catches **fine, tiny** changes, but also reacts to noise, lighting and slight misalignment.

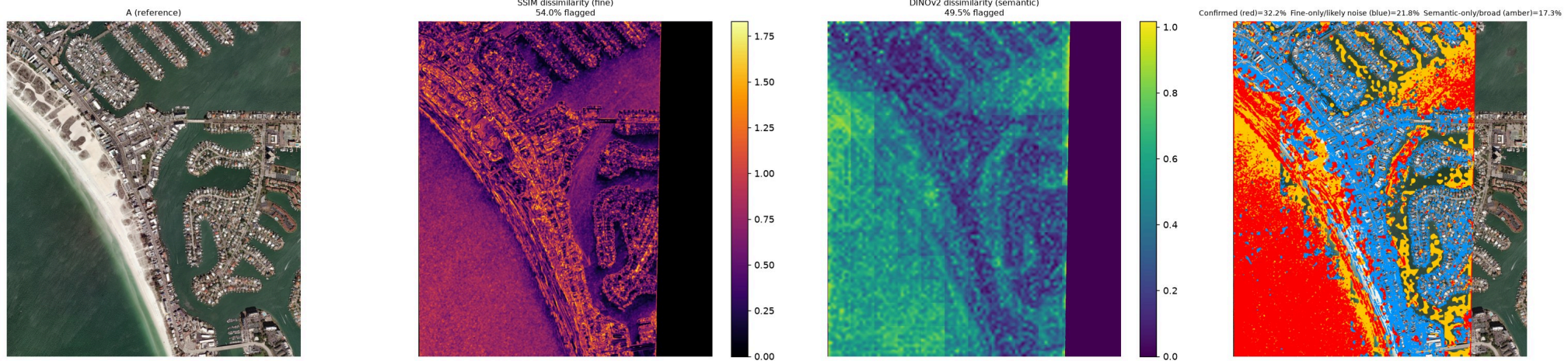
DINOv2 (AI, meaning-level)

Understands **what** is in the scene and is robust to noise, but coarser.

Combined

Flag a spot as real change **only where both agree**. That gives a cleaner signal than either one alone.

Maxar: 2023-11-04 (pre) vs 2024-10-10 (post-Milton) — combined SSIM (fine) + DINOv2 (semantic) change detection



Red = both agree (confirmed) **Blue** = SSIM only (likely noise) **Amber** = DINOv2 only (broad shift, e.g. water color)

WHAT I CHANGED · 2 (RESULTS)

Combined signal: results on all three image pairs

IMAGE PAIR	SSIM FLAGGED	DINOv2 FLAGGED	CONFIRMED	LIKELY NOISE
Maxar pre → post	54.0%	49.5%	32.2%	21.8%
NOAA Helene → Milton	42.4%	40.4%	25.6%	16.8%
Sentinel-2 pre → post	28.0%	51.3%	24.9%	3.1%

Requiring agreement removes **17–22%** of the image area that SSIM alone would have flagged as change but is most likely noise (Maxar, NOAA).

Honest caveat: the automatic threshold (Otsu) is still loose on busy scenes, so "confirmed" currently shows neighborhood-level change rather than tight damage spots. Next: a stricter top-10–15% threshold.

3 / 3

sources agree the **Gulf-facing beach & dune line** took the heaviest change, while interior canal neighborhoods stayed more stable.

Compare each building, not the whole scene

Adapted from **Microsoft's HASTE** disaster-response platform:

- 1 **Get every building's outline.** I pulled **4,382** building footprints for Treasure Island from OpenStreetMap.
- 2 **Crop to one building** (plus a little context) in the before and after images.
- 3 **Describe only the pixels inside its outline** with DINOv2, producing one "fingerprint" per building.
- 4 **Compare the before and after fingerprints** to get a change score for that building.

Why this solves it

A small change on a roof is now a **big share of that building's own analysis**, instead of being averaged away across a random tile with water, roads and other houses.

Validated on a test

On a synthetic image built from the real footprints, ~14 deliberately altered buildings scored **0.13–0.76**. Unchanged buildings scored ≈ 0.0 , a clean separation.

NEXT STEP

From "how much changed" to "how damaged"

A change score says *something* changed. Responders need a **damage category**.

I've scaffolded a classifier trained on **xBD**, the standard building-damage dataset (~850k labeled buildings across 19 disasters, including hurricanes).

No damage

Minor

Major

Destroyed

It uses the same per-building fingerprint as the previous slide, plus a simple linear classifier (the same recipe HASTE uses).

Status

- **Built & tested:** full pipeline verified end-to-end on a synthetic xBD-format dataset
- **Plugged in:** once trained, the per-building script automatically adds a damage class for every building
- **Pending:** xBD download (requires registration at xview2.org, ~10 GB)

WHERE EVERYTHING LIVES

Code & outputs map

FILE	WHAT IT DOES	OUTPUT
<code>compare_images.py</code>	Alignment + SSIM baseline	<code>analysis_output/<pair>.png</code> , <code>metrics.json</code>
<code>dinov2_compare.py</code>	Fix 1: full-frame, overlapping-tile DINOv2 map	<code>analysis_output/<pair>_dinov2.png</code>
<code>combined_compare.py</code>	Fix 2: SSIM + DINOv2 agreement	<code>analysis_output/<pair>_combined.png</code>
<code>footprint_compare.py</code>	Fix 3: per-building scoring (HASTE-style)	per-building GeoJSON (needs geo-imagery)
<code>footprints/...buildings.geojson</code>	4,382 OSM building outlines	input
<code>train_xbd_probe.py</code>	Next: damage-category classifier on xBD	<code>models/xbd_dinov2_probe.joblib</code>
<code>reports/...Progress_Report.pdf</code>	Detailed written report for this round	n/a

Repository: github.com/sai0215/ImageClassTreasureIsland

Next steps

- 1 Georeferenced imagery.** The current images are flat previews with no map coordinates, so building outlines can't be placed on them yet. Getting GeoTIFF versions unlocks the per-building run on real data.
- 2 Train on xBD.** Download the dataset and train the damage classifier, giving no / minor / major / destroyed for each building.
- 3 Tighten the combined map** with a stricter percentile threshold so it highlights specific damage spots.

Summary

- Found and fixed a hidden crop that discarded up to ~50% of each image
- Replaced a blurry 16×16 map with a full-resolution one
- Added a cross-checked SSIM + DINOv2 signal
- Moved to **per-building** analysis, so small damage now stands out

THANK YOU

Questions & discussion

Thank you to Anwesha and the team for the guidance and resources. I'd be glad to keep building on this work, including as part of my MSAI thesis.